

Программирование и алгоритмизация

(МИСиС, осень-зима 2025)

Полевой Дмитрий Валерьевич
к.т.н., доцент КиК

teams: 8url2s2

e-mail: dvpsun@gmail.com

tg: @dvpsun

ваши вопросы

Классы ввода/вывода

- `ostream`
 - поток вывода (класс)
 - глобальные объекты `cout`, `cerr`, `clog`
- `istream`
 - поток ввода (класс)
 - глобальный объект `cin`

Проблема перегрузки операций ВВОДА И ВЫВОДА

- левым аргументом является экземпляр класса стандартной библиотеки
- модификация класса левого аргумента невозможна или нежелательна

Перегрузка ввода/вывода

- **ВЫВОД**

`ostream& operator<<(ostream&, T)`

или

`ostream& operator<<(ostream&, const T&)`

- **ВВОД**

`istream& operator>>(istream&, T&)`

Оператор вывода (пример)

```
bool T::writeTxt(ostream& ostr) const;
```

```
ostream&
```

```
operator<<(ostream& ostr, const T& rhs) {  
    rhs.writeTxt(ostr);  
    return ostr;  
}
```

Друзья

- метод
 - имеет доступ к закрытой части объявления
 - находится в области видимости класса
 - должен вызываться для экземпляров класса (имеется указатель `this`)
- друг (`friend`-функция или `friend`-класс)
 - имеет доступ к закрытой части объявления

friend-функция

- указывается в объявлении класса, другом которого она является
- может указываться в любой секции (`public`, `protected`, `private`)

Практика использования `friend`

использовать `friend` не рекомендуется

допустимо

- реализация сильно связанных концепций
- удовлетворение обоснованных требований по оптимизации

Friend-функция (пример)

```
class Matr;  
  
class Vect {  
    friend Vect operator*(const Matr&, const Vect&);  
};  
  
class Matr {  
    friend Vect operator*(const Matr&, const Vect&);  
};  
  
Vect operator*(const Matr& lhs, const Vect& rhs) {  
    //...  
}
```

ваши вопросы

Константность (`const`)

- замена препроцессора
- строгий контроль типов
- защита данных
- оптимизация

Константность и защита данных

- инициализация в процессе выполнения
- изменение не предусмотрено
- компилятор пресекает попытки
потенциального изменения
(проверка присваиваний и модификации)

Константные переменные

- не изменяются после определения

пример:

```
const ptrdiff_t size(vec.size());
```

Константные параметры

- не изменяются внутри функции

пример:

```
int min(const int lhs, const int rhs)
ostream& write(ostream& ostrm, const T& val)
```

Константные поля

- не изменяются после создания объекта
- инициализируются
 - в списке инициализации
 - умолчательным инициализатором

Константные поля (пример)

```
class FixedArr {  
    public:  
        FixedArr(const int size);  
    private:  
        const int size_{0};  
        ///...  
};  
  
FixedArr::FixedArr(const int size)  
    : size_(size) {  
    ///...
```

Константные методы

- сохраняют логическую константность
 - не изменяют состояния полей кроме mutable
- могут вызываться для константных экземпляров
- отличаются от неконстантных методов с тем же набором параметров

Константные методы (пример)

```
class FixedArr {  
    public:  
        //...  
        int size() const;  
        //...  
};  
  
int FixedArr::size() const {  
    //...
```

Поля-ссылки

- сильно связанные данные (классы)
- инициализируются в списке инициализации
- блокируют создание умолчательного конструктора

Поля-ссылки (пример)

```
class Point {  
    public:  
        explicit Point(const Axes& axes);  
    private:  
        const Axes& axes_  
        ///...  
};  
  
explicit Point::Point(const Axes& axes)  
    : axes_(axes)  
    ///...
```

ваши вопросы

Чем отличается
копирование
от
присваивания?

Отличие копирования от присваивания

```
T::T(const T&obj)
```

```
// состояния ещё нет
```

```
T& operator=(const T&obj)
```

```
// состояния уже есть
```

```
// при "проблемах" хочется сохранить
```


Исключения и operator=

```
T& T::operator=(const T& obj) {  
    // возможны проблемы  
    // как "откатить изменения"?  
    return *this;  
}
```

Помним про $x=x$ (присваивание самому себе)

```
T& T::operator=(const T& obj) {  
    if (this != &obj) {  
        // ВОЗМОЖНЫ ПРОБЛЕМЫ  
        // как "откатить изменения"?  
    }  
    return *this;  
}
```

Cxema copy-and-swap

```
T& T::operator=(const T& obj) {  
    if (this != &obj) {  
        swap(*this, T(obj))  
    }  
    return *this;  
}
```

Схема copy-and-swap (не оптимально)

```
T& T::operator=(T obj) {  
    swap(*this, obj)  
    return *this;  
}
```

ваши вопросы

Повторное использование кода

- уменьшение «стоимости» создания и поддержки ПО
 - разработка, отладка и тестирование
 - сопровождение
 - модификация
- эффекты «массовости»

Реализация повторного использования кода

- модульность – использование готовых компонент (классы, библиотеки, функции)
- создание новых классов на основе существующих
 - инстанцирование (шаблоны)
 - наследование

Проблема повторного использования алгоритмов

- один и тот же алгоритм может обрабатывать данные разных типов

пример:

```
int minimum(const int lhs, const int rhs) {  
    return (lhs < rhs) ? lhs : rhs;  
}
```


Как использовать алгоритм с разными типами?

- «сору-past» – усложнение поддержки кода (размножение «одинаковых сущностей», внесение новых ошибок)
- макросы – усложнение отладки и поддержки

пример:

```
#define min(ls, rs) ((ls < rs) ? ls : rs)
```

Как правильно использовать
алгоритм с разными типами?

написать шаблонный код

Ключевые слова

- `template`
 - описание параметризации шаблонна
- `class` и `typename`
 - именованение параметра

пример:

```
template<class T>
```

или

```
template<typename T>
```

Структура шаблонного кода

```
template<"список параметров">  
"объявление"
```

```
template<"список параметров">  
"определение"
```

Шаблон (template)

- в определении (класса или функции) используется один или несколько неизвестных (именованных) типов
- типы должны быть известны в момент компиляции программы

Преимущества шаблонов

- использование типа при определении функции или класса
- автоматическая генерация кода
- зависит только от тех параметров типа, которые использует
- оптимизация программ

Инстанцирование шаблона

- генерация объявления функции (типа) по шаблону функции (типа) и аргументам шаблона
- осуществляется компилятором на этапе компиляции
- сгенерированный код подчиняется стандартным правилам

Шаблонная функция

- определяется шаблоном и набором параметров

пример:

```
template<typename T>  
const T& min(const T& lhs, const T& rhs) {  
    return (lhs < rhs) ? lhs : rhs;  
}
```


Шаблонный класс (тип)

- определяется шаблоном и набором параметров
- при использовании одного набора параметров генерируется один и тот же тип (эквивалентные типы)

Шаблонный тип (пример)

```
template<class T>
class Stack {
    public:
        Stack();

        ...

        T& getTop();
    private:
        T* pData_{nullptr};

        ...
};
```

Использование шаблонного типа

- указать параметры шаблона
- использовать как «обычный тип»

пример:

```
Stack<int> st;  
st.push_back(0);
```

ваши вопросы

Параметры шаблонов

- типизированный параметр
 - тип, м.б. тоже шаблонный
- нетипизированный параметр
 - константа времени компиляции
- параметров м.б. несколько

Шаблоны членов и методов типа

- шаблонный тип может иметь шаблонные члены и шаблонные методы
- нешаблонный тип может иметь шаблонные методы

Пример шаблонного метода

```
template<typename T>
class Agr {
    ...
    template<typename U>
    void mult(const U& rhs);
    ...
};
```

Нетипизированный параметр

- известное на стадии компиляции
целочисленное значение
(литерал или выражение)
- в т.ч. адрес объекта, адрес функции с
внешней компоновкой, выражение от типа
и т.д.

Нетипизированный параметр (пример)

```
template<class T, size_t N>
class Arr {
    ...
    private:
        T data_[N]{T()} ;
};
```

```
Arr<int, 128> members;
```

Хочется вещественный нетипизированный параметр?

- проверить что «очень надо»
 - помнить об отсутствии «точного» представления
- использовать `std::ratio`
 - шаблон арифметики рациональных чисел
времени компиляции

Типизированный параметр

- фундаментальный тип
- пользовательский тип
- шаблонный тип
- в т.ч. производный

Типизированный параметр (пример)

```
template<class T, size_t N>  
class Arr {  
    ...  
    private:  
        T data_[N] {T()} ;  
};
```

```
Arr<int, 128> members;
```

Проверка

- при определении шаблона
 - ошибки, которые можно обнаружить вне зависимости от конкретного набора аргументов
- при инстанцировании шаблона
 - ошибки, которые имеют отношение к использованию параметров

Параметры шаблона функции

- могут быть выведены компилятором, если однозначно определяются списком аргументов
- могут быть указаны явно

пример:

```
int minVal (minimum<int> (a, b) );
```

Параметры шаблона класса

- должны указываться явно, т.к. никогда не выводятся

пример:

```
vector<int> marks;
```

- для вывода списка параметров м.б. использована генерирующая экземпляры шаблонного класса функция

пример:

```
make_pair(a, b)
```

Автоматическая генерация экземпляров шаблонного класса

```
template<typename F, typename S>  
struct pair;
```

```
template<typename F, typename S>  
pair<F, S> make_pair(const F& f, const S& s)  
{  
    return pair<F, S>(f, s);  
}
```


Перегрузка функций

- можно объявлять
 - несколько шаблонов с одним именем
 - комбинацию шаблонов и обычных функций с одним именем
- выбор функции
 - явный квалификатор (параметры шаблона)
 - разрешение вызова

Перегрузка функций (пример)

```
template<typename T>  
T sqr(const T& v);
```

```
template<typename T>  
complex<T> sqr(const complex<T>& z);
```

```
double sqr(const double v);
```

Параметр шаблона по умолчанию (пример)

```
template<class T> class Str; //< строки  
template<class T> class Cmp; //< сравнитель символов
```

```
template<class T>  
int compare(const Str<T>& lhs, const Str<T>& rhs);  
  
template<class T, class C>  
int compare(const Str<T>& lhs, const Str<T>& rhs);
```

или

```
template <class T, class C = Cmp<T>>  
int compare(const Str<T>& lhs, const Str<T>& rhs);
```

ваши вопросы

Размещение шаблонного кода

- определение шаблона д.б. доступно в точке инстанциирования
- размещайте весь шаблонный код в заголовочных файлах

Определение функции (пример)

```
// in service.h  
template<typename T>  
const T& minimum(const T& lhs, const T& rhs) {  
    ...  
}
```

Определение невстраиваемых методов класса (пример)

```
// in arr.h  
template<typename T>  
class Arr {  
    public:  
        int size() const;  
  
    ...  
};
```

```
// in arr.h  
template<typename T>  
int Arr<T>::size() const {  
    ...  
}
```

Специализация

- версия шаблонного кода для конкретного набора параметров
- позволяет реализовывать выбор оптимальной реализации для заданного набора параметров
- некоторые специализации сильно меняют реализацию (`std::vector<bool>`)

Псевдоним шаблона (template alias)

```
using Type = type_definition;
```

- определить новое (короткое) имя типа
- «привязать» (bound) часть параметров

Псевдоним шаблона (пример)

```
// Стандартный вектор и спец. аллокатор  
template<class T>  
using Vec = std::vector<T, SpecAlloc<T>>;  
  
Vec<int> marks{5, 4, 3, 2, 1};
```

Тип выражения (decltype)

decltype (E)

- тип имени или выражения E
- может быть использован в объявлениях

пример:

decltype (x * y)

Вывод типа из инициализатора

```
auto v ("выражение") ;
```

- тип переменной определяется при компиляции
- тип может быть не известен, либо сложен в написании

пример:

```
auto v(arr.begin()) ;
```

Пример использования auto

```
template<class T, class U>
void multiply(const vector<T>& vt,
             const vector<U>& vu) {
    // ...
    auto tmp(vt[i] * vu[i]);
    // ...
}
```

Использование auto (рекомендации)

без «фанатизма»

- польза должна превышать вред
- ПОМНИТЬ
 - не уверен — не используй
 - тип есть всегда
 - размер области видимости играет роль

Проблема вывода типа возвращаемого значения

```
template<class T, class U>  
??? mul(T x, U y) {  
    return x*y;  
}
```

Суффиксный синтаксис возвращаемого значения

auto `f ("параметры") -> T`

- `auto` указывает вывод типа
- тип возвращаемого значения размещается после списка аргументов

Пример вывода типа возвращаемого значения

```
template<typename T, typename U>  
auto mul(const T& x, const U& y) ->  
    decltype(x * y) {  
    return x * y;  
}
```

Шаблоны с переменным числом параметров (Variadic Templates)

```
template<typename ... Args>
```

- найдите, что есть и такое
- используйте стандартные типы и функции
- дальше разберетесь сами 😊

ваши вопросы

Слабая типизация

- тип обеспечивает интерфейс, ожидаемый шаблоном
- требуется доступность вызываемых функций для типов, параметризующих шаблоны

Слабая типизация (пример)

```
template<typename T>  
const T& min(const T& lhs, const T& rhs) {  
    return (lhs < rhs) ? lhs : rhs;  
}
```

Параметрический полиморфизм или полиморфизм времени компиляции

- реализуется с помощью механизма шаблонов
- алгоритм выражается один раз и используется многократно со множеством типов

Генерируемый код

- инстанцирование должно иметь место только для используемого кода
- для каждого уникального набора параметров генерируется свой код
- возможно «разбухание» программы

Сообщения об ошибках в шаблонном коде

- требуют внимательного разбора
- часто связаны с передачей недопустимых параметров

STL (Standard Template Library)

- шаблонная часть стандартной библиотеки
- содержит много полезных и качественно реализованных типов и функций

ваши вопросы

Кортеж

- последовательность конечного числа элементов
- пустое множество — кортеж
(с нулевым количеством элементов)
- для каждого кортежа $(a_1, \dots, a_n) = T$,
множество $(a, a_1, \dots, a_n) = \{a, \{a, T\}\}$ также
является кортежем

Упорядоченная пара

- кортеж длины 2

$$\langle e_1, e_2 \rangle = \{e_1, \{e_1, e_2\}\}$$

- сравнение на равенство

$$\langle x_1, x_2 \rangle = \langle y_1, y_2 \rangle \Leftrightarrow x_1 = y_1 \wedge x_2 = y_2$$

в чем сложность реализации
шаблонного типа кортеж?

std::pair

```
template<class T1, class T2>  
struct pair;
```

поля `first` и `second`

производящая функция `make_pair`

пример:

```
auto el(make_pair(a, f(b)));
```

Вариативный шаблон (variadic template)

`template<typename . . . Args>`

- шаблон с переменным числом аргументов
- используется для создания
 - рекурсивных шаблонных типов
 - рекурсивных шаблонных функций

`std::tuple`

- реализуется через variadic template

пример:

```
auto t(make_tuple(1, 3.14, str));
```


Доступ к элементам кортежа

`get<N>()` - по индексу элемента

`get<T>()` - по типу (для уникальных)

код доступа генерируется при компиляции

пример:

```
auto i(get<1>(t)); // по индексу
```

```
auto j(get<double>(t)); // по типу
```

ваши вопросы

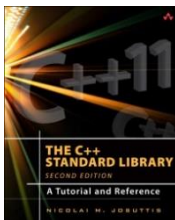
Литература (про STL)



Остерн М.Г.
Обобщенное программирование и STL



Степанов А., Роуз Э.
От математики к обобщенному программированию



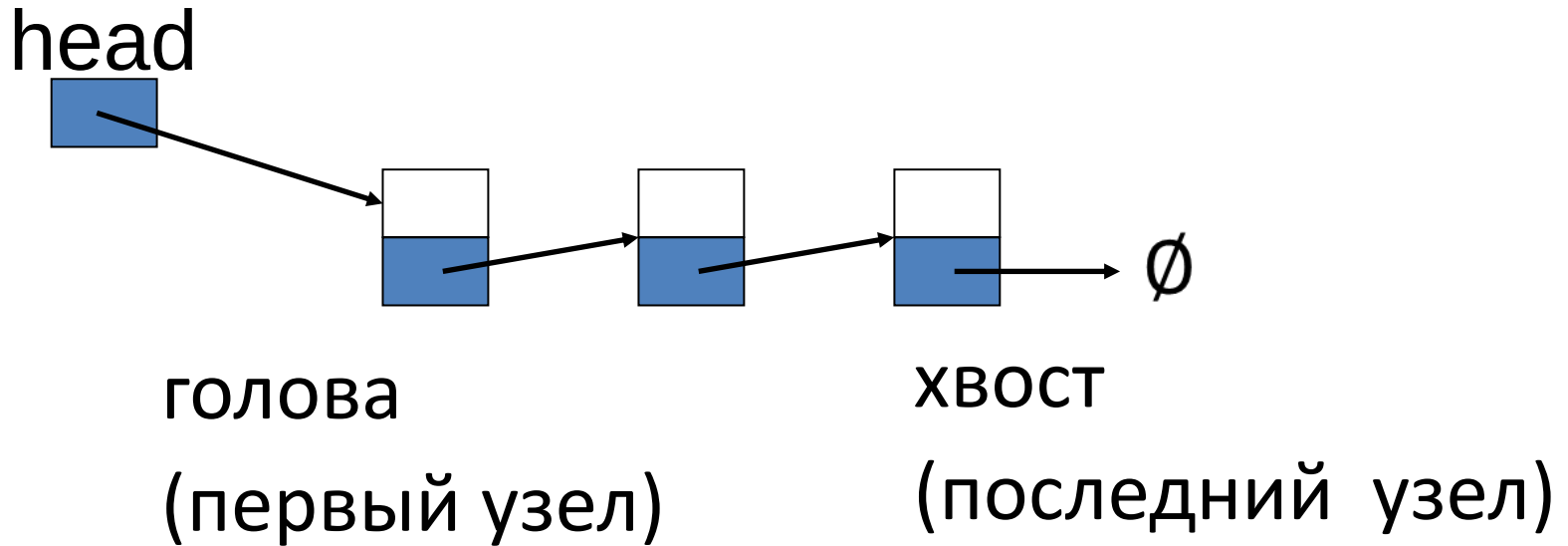
Josuttis N.
The C++ Standard Library. A Tutorial and Reference

ваши вопросы

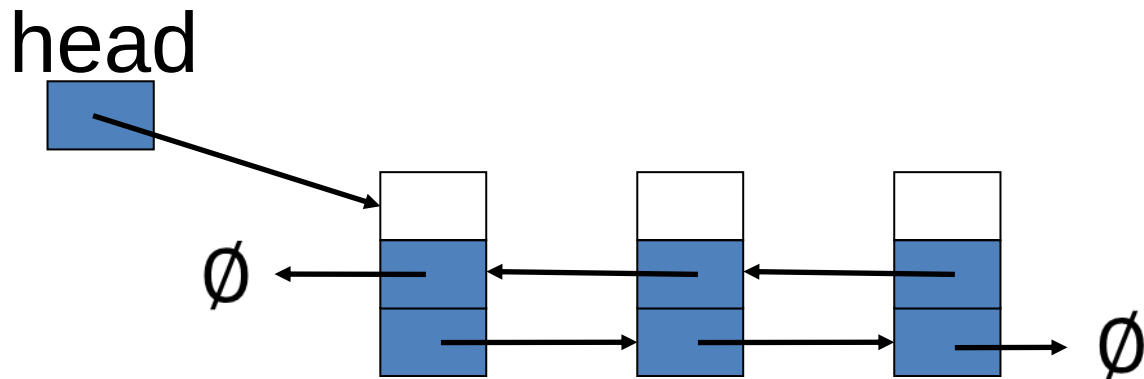
Список (структура данных)

- последовательный доступ к элементам
- состоит из узлов
- узел содержит
 - собственно данные
 - связи с другими узлами

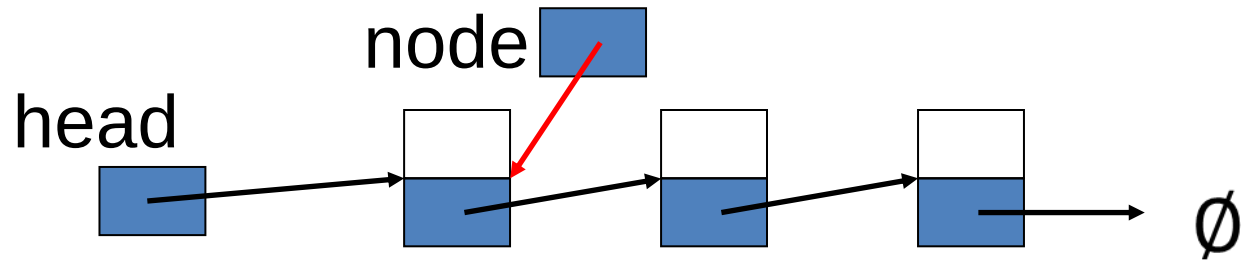
Односвязный список (однонаправленный)



Двусвязный список (двунаправленный)

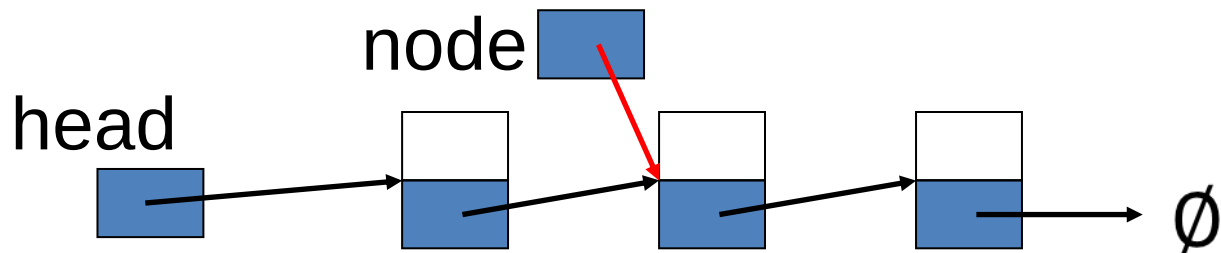


Навигация по линейному списку



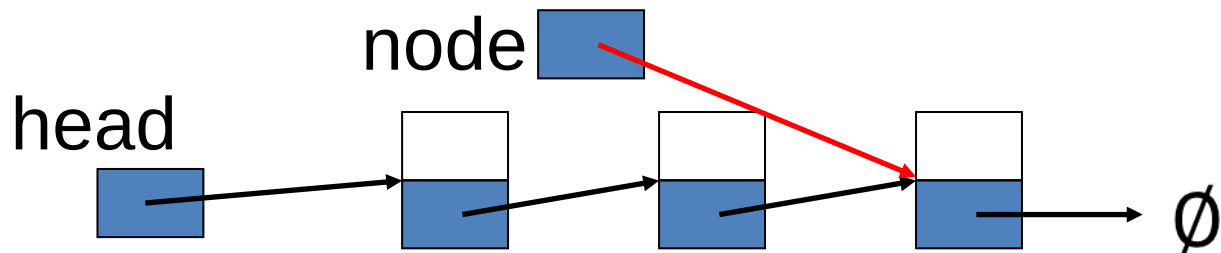
```
node = head;
```


Навигация по линейному списку



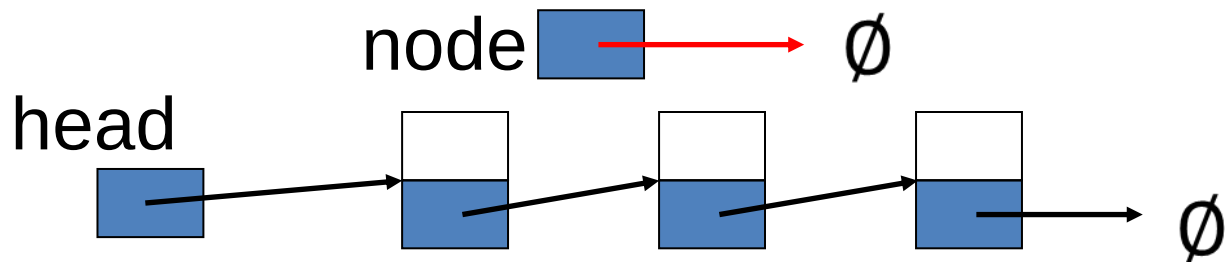
```
if (nullptr != node) {  
    node = node->next_;  
}
```

Навигация по линейному списку



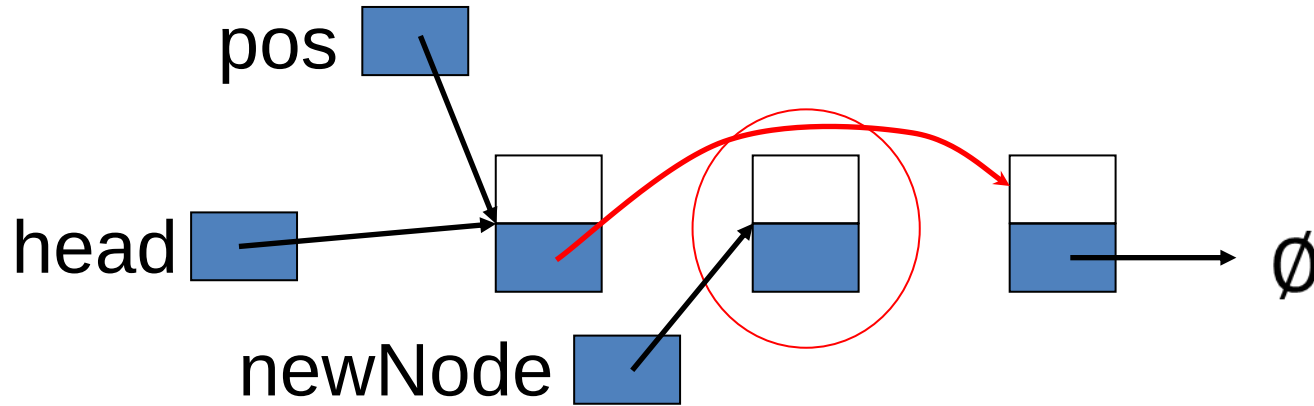
```
if (nullptr != node) {  
    node = node->next_;  
}
```

Навигация по линейному списку



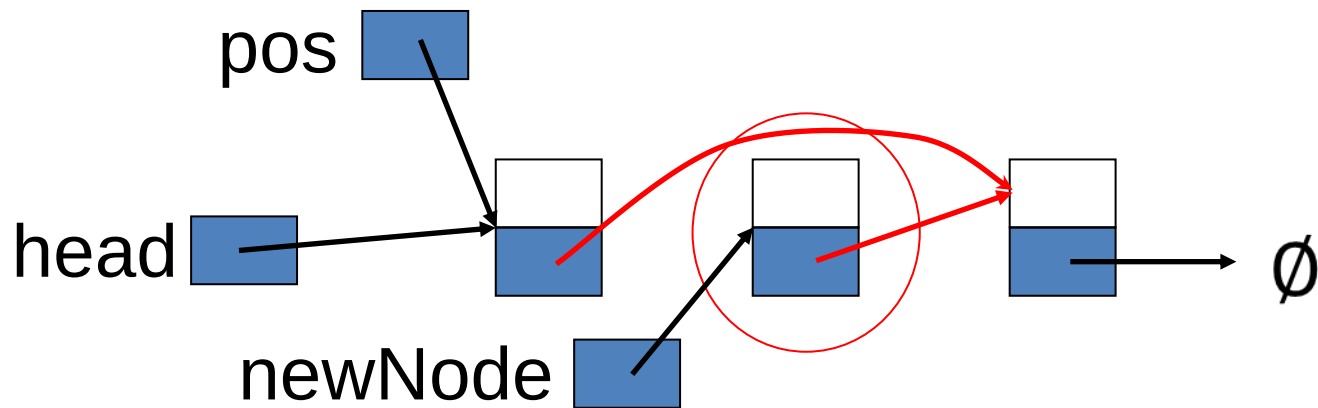
```
if (nullptr != node) {  
    node = node->next_;  
}
```

Вставка узла



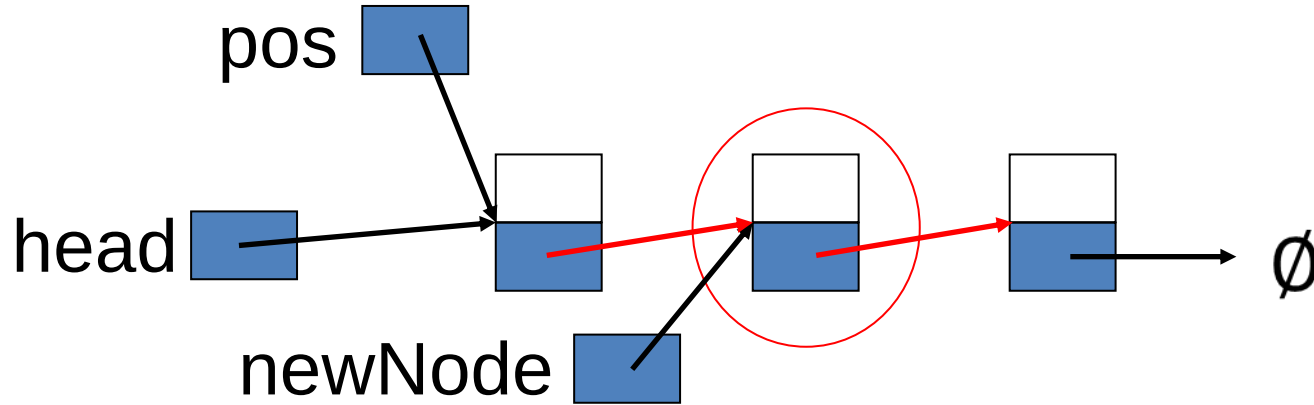
```
newNode = new Node;  
newNode->next_ = pos->next_;
```

Вставка узла



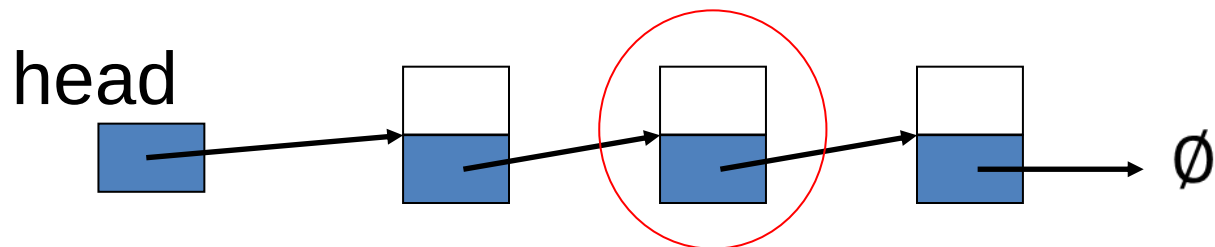
```
newNode->next_ = pos->next_;
```

Вставка узла

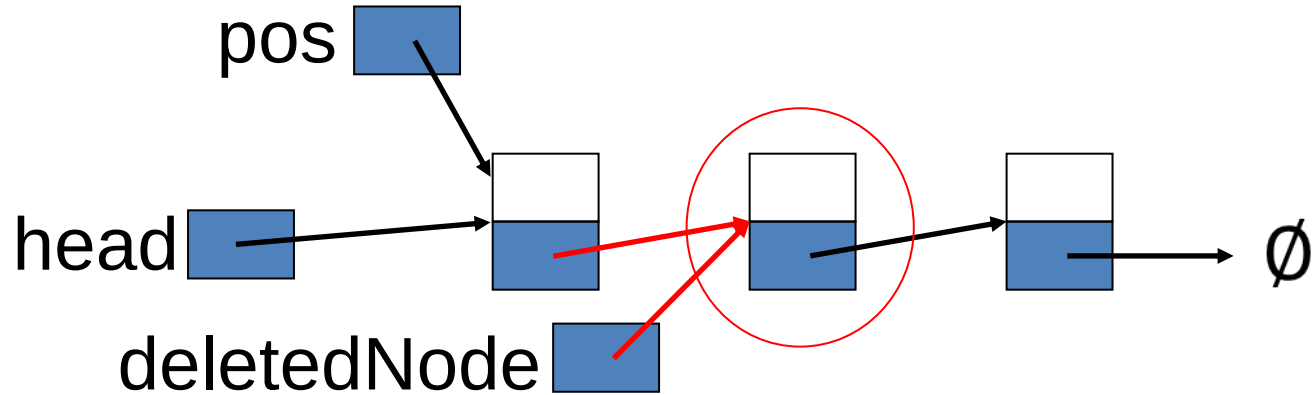


```
pos->next_ = newNode;
```

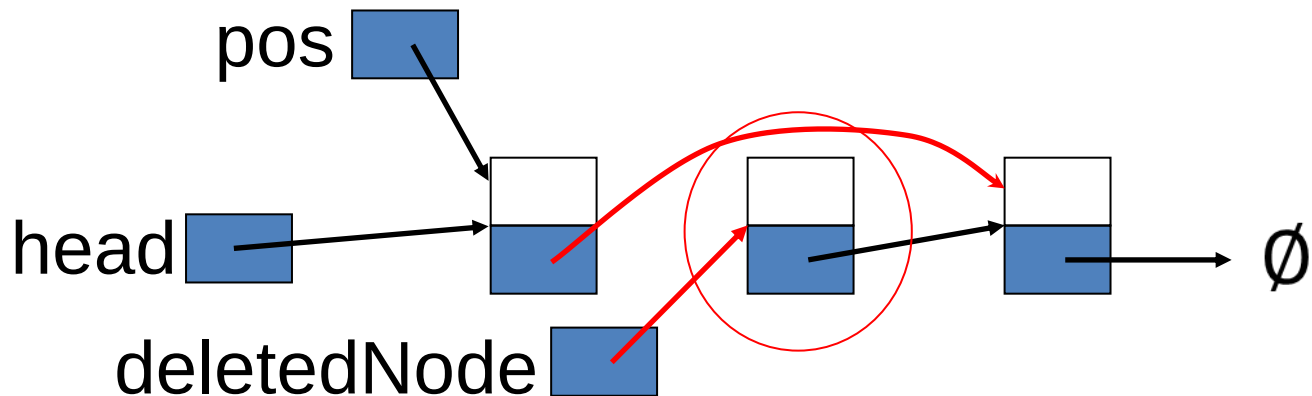
Удаление узла



Удаление узла

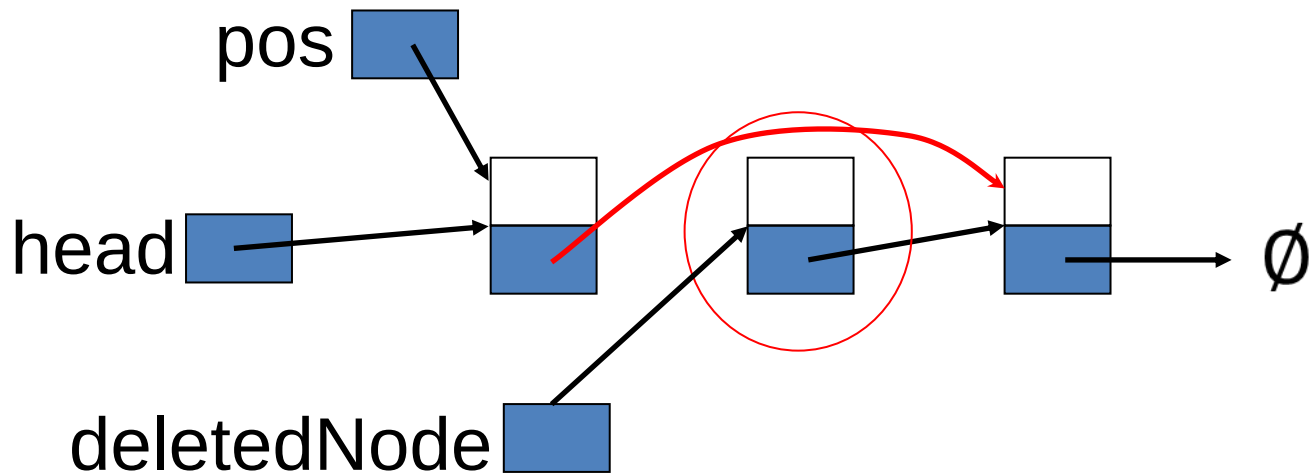


Удаление узла



```
pos->next_ = deletedNode->next_;
```

Удаление узла



```
delete deletedNode;
```

Уничтожение списка

- удаление узлов до полного исчезновения

```
while (!empty()) {  
    // delete head;  
}
```

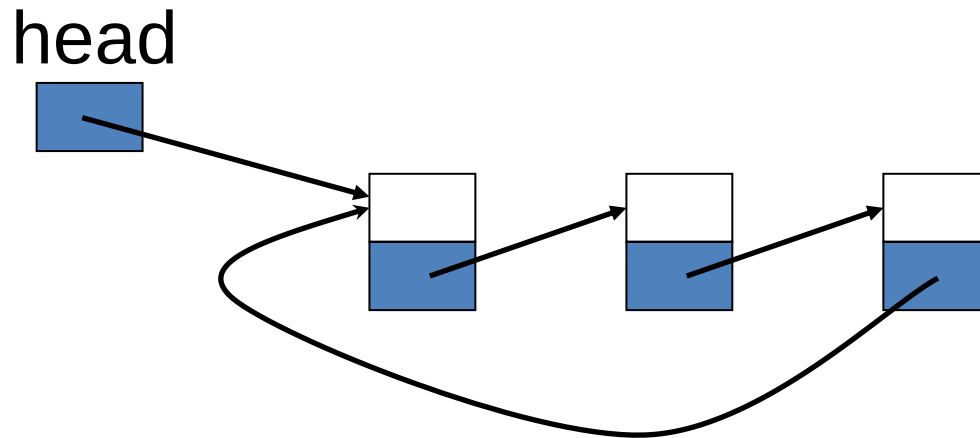
Плюсы списков

- простое динамическое добавление и удаление элементов
- размер ограничен только объёмом памяти и разрядностью указателей
- элементы не перемещаются в памяти

Минусы списков

- накладные расходы памяти на связи
- «долгое» обращение к элементу по индексу
- «долгое» выделение узла
- элементы списка могут быть расположены в памяти разреженно
(фрагментация памяти, плохо кэшируется)

Кольцевой список



ваши вопросы